Lecture 3

BLAST and Sequence Similarity

From biological intuition to command-line analysis

Computing for Data-Driven Biology · 5BI00A
Master's Programme in Bioinformatics · Umeå University

Session Aims & ILO Links

1	Explain what sequence similarity is and why it is biologically meaningful	ILO 5
2	Describe how BLAST works — seeding, extension, scoring, and E-values	ILO 1
3	Interpret a BLAST results table including E-values, bit scores, and % identity	ILO 7
4	Distinguish the five BLAST programmes and select the appropriate one	ILO 5
5	Run BLAST via web interface, NCBI API, and locally on HPC2N via Slurm	ILO 7, 9
6	Commit BLAST inputs, scripts, and outputs to Git with meaningful messages	ILO 12

This lecture follows directly from the database session. Have your TP53 sequence from Lecture 2 ready.

Opening Poll

Have you used BLAST before?

- Yes — I know what it does
- Yes — but I'm not sure how it works
- I've heard of it but not used it
- No — completely new to me

Also: word cloud — in one word, what do you think BLAST stands for?

(Answer: Basic Local Alignment Search Tool)

Part 1

Sequence similarity — the biological foundation

Similarity Implies Common Ancestry

Sequences that are similar are likely related by evolution — they are homologous.
Homologous sequences often share biological function because structure is conserved by natural selection.

🧬 Orthologues

Genes in different species descended from the same ancestral gene via speciation. Usually retain the same function. This is the basis of functional annotation transfer across species.

🔀 Paralogues

Genes within the same genome related by gene duplication. Often diverge in function — one copy may retain the ancestral role while the other acquires a new one. Be cautious about annotation transfer between paralogues.

⚠️ **Similar sequence ≠ identical function.**
The top BLAST hit is a hypothesis about function, not a fact. The strength of that hypothesis depends on E-value, % identity, query coverage, and the reliability of the subject sequence's own annotation.

Part 2

How BLAST works — a conceptual overview

The Problem BLAST Solves

You have one query sequence. The database has billions. How do you find all similar sequences quickly?

Naïve approach

Compare every residue of the query to every residue of every sequence in the database. Guaranteed to find the optimal alignment — but computationally impossible for large databases.

BLAST — heuristic approach

Use short-word seeds to rapidly identify candidate matches, then only extend full alignments from those seeds. Fast enough for billions of sequences. Does not guarantee finding the optimal alignment — but finds the biologically important ones.

Heuristic = fast and good enough. Not exhaustive. Understanding this limitation is part of interpreting BLAST results responsibly.

Seed and Extend: The Two-Stage Strategy

1 Seeding

Break the query into short overlapping words (k-mers). Find exact or near-exact matches to these words in the pre-indexed database. Fast — uses a lookup table. Each match is a seed.

2 Extension

Extend each seed in both directions, one residue at a time, as long as the alignment score keeps improving. Stop when the score drops too far below the best seen. The result is a High-Scoring Pair (HSP).

3 Reporting

Rank all HSPs by score. Calculate E-values. Report those above the significance threshold. Multiple HSPs for the same subject sequence may be merged.

Word size k: typically 11 nt for blastn, 3 aa for blastp. Smaller k = more sensitive but slower. Larger k = faster but may miss distant homologues.

Scoring: Substitution Matrices

Protein BLAST scores are not simply +1 for match, −1 for mismatch. Each amino acid pair has its own score based on how often they are observed to substitute for one another in real, related proteins.

BLOSUM matrices

Derived from observed substitutions in conserved blocks of related proteins.

BLOSUM80	Closely related sequences (< 80% identity used)
BLOSUM62	Moderately diverged — DEFAULT for most searches
BLOSUM45	Distantly related sequences — more sensitive

What the number means

BLOSUM62 is derived from sequences sharing ≤62% identity. The lower the number, the more diverged the reference sequences — and the better the matrix is at detecting distant relationships.

Conservative substitutions (e.g. Leu→Ile, similar chemistry) score positively. Radical substitutions score negatively. This is why protein BLAST is more sensitive than nucleotide BLAST for distant homologues — amino acid chemistry is informative in a way that nucleotide identity alone is not.

In practice: use BLOSUM62 for most searches. The Comparative Genomics course covers matrix selection in depth.

The E-value: The Most Important Number in Your Results

E-value = how many times would you expect to find a match this good (or better) by chance in a database of this size?

`0.0 (≈ 1e-180+)`	Certain	Astronomically unlikely to be coincidental. Essentially certain homologue.
`1e-50`	Very strong	Very strong hit. Highly conserved sequence. Almost certainly true homologue.
`1e-10`	Strong	Strong hit. Likely homologue. Worth investigating.
`1e-5`	Moderate	Moderate. Generally accepted threshold for annotation transfer. Check carefully.
`0.01`	Weak	Weak hit. Treat with scepticism. May be real or may be noise.
`≥ 1.0`	Noise	Expected to occur by chance. Almost certainly not biologically meaningful.

⚠️ E-value depends on database size. The same alignment gives a lower E-value against a small database than a large one. Always report which database you searched.

The Five BLAST Programmes

blastn	Query: Nucleotide	DB: Nucleotide	Find similar DNA/RNA sequences. Identify a species from a sequenced fragment (e.g. 16S rRNA, COI barcode). Fast but less sensitive for distant homologues.
blastp	Query: Protein	DB: Protein	Find similar protein sequences. Most sensitive for distant evolutionary relationships. Default choice for functional annotation.
blastx	Query: Nucleotide (translated)	DB: Protein	Find protein matches for an unannotated nucleotide sequence. Translates in all 6 reading frames. Useful when you have a new genomic sequence but no gene model.
tblastn	Query: Protein	DB: Nucleotide (translated)	Find unannotated genomic regions that encode a protein of interest. Useful for identifying new gene models in an unannotated genome.
tblastx	Query: Nucleotide (translated)	DB: Nucleotide (translated)	Most sensitive nucleotide-to-nucleotide search (via protein translation). Slow. Used for finding distant nucleotide homologues where blastn fails.

Interpreting the BLAST Results Table

Description

Name of the matching database sequence (subject)

E value

Expect value — how likely this match is by chance. Your primary significance indicator.

Per. ident %

Percentage of identical residues in the aligned region. Always check alongside query cover.

Query cover

Fraction of your query sequence covered by the alignment. 10% coverage at 90% identity ≠ 90% coverage at 90% identity.

Bit score

Database-size-independent alignment quality score. Use for comparing across different searches.

Accession

Database accession number. Use to fetch the full sequence or annotation.

Break — 10 minutes

Back at: _____

After the break: three ways to run BLAST · web → API → local HPC · hands-on exercise

Part 3

Three ways to run BLAST — web · API · local HPC

Three Ways to Run BLAST

🌐 Web Interface

blast.ncbi.nlm.nih.gov

✓ Interactive, visual results viewer
✓ No setup required
✓ Easy to share with collaborators

✗ Not reproducible (parameters not recorded)
✗ Cannot automate
✗ Not scalable beyond a handful of sequences

⌨️ NCBI API

curl / Entrez utilities

✓ Parameters captured in code
✓ Integratable with other tools
✓ Documented and commitable to Git

✗ Asynchronous (submit → wait → retrieve)
✗ Rate limited by NCBI
✗ Slow for many sequences

🖥️ Local BLAST on HPC

BLAST+ module on Kebnekaise

✓ Fastest for large-scale searches
✓ Full parameter control
✓ Fully reproducible via Slurm scripts

✗ Requires local database copy
✗ Setup time
✗ Must manage database updates yourself

Exercise: BLAST Three Ways — TP53 Revisited

You retrieved the TP53 sequence in Lecture 2. Now you will BLAST it three different ways, building a complete documented and committed workflow.

Part A — **Setup**

Create lecture03-blast directory · copy TP53_protein.fasta from Lecture 2 · initialise Git · first commit

Part B — **Web BLAST**

blastp vs SwissProt · interpret top 5 hits · find conservation range · identify most conserved regions

Part C — **NCBI API**

Submit blastp via curl · capture RID · retrieve results · inspect with grep · commit with full metadata

Part D — **Local HPC via Slurm**

Load blast+ module · write job script · sbatch · monitor with squeue · parse tabular output · sort by E-value

Part E — **Compare approaches**

Complete the comparison table in your README · commit final observations · discuss with your neighbour

Full commands and questions in your handout. Extension task: BLAST Arabidopsis ubiquitin — compare conservation breadth to TP53.

Exercise: Setup and Web BLAST

Part A — Setup

```
$ cd ~/course && mkdir -p lecture03-blast && cd lecture03-blast
$ git init
$ cp ../lecture02-databases/TP53_protein.fasta .
$ git add TP53_protein.fasta && git commit -m "initial commit: lecture03 blast, TP53 query from lecture02"
```

Part B — Web BLAST (questions to answer)

Go to blast.ncbi.nlm.nih.gov → Protein BLAST → Database: Swiss-Prot

1. What are the top 5 hits and which organisms do they come from?

2. At what % identity do non-mammalian vertebrate hits appear?

3. Can you find a plant or fungal homologue? What is the E-value and annotation?

4. Which regions of TP53 are most conserved (look at the alignment)?

Exercise: API and Local HPC BLAST

Part C — NCBI API (asynchronous: submit → wait → retrieve)

```
# Submit blastp job — replace your@email.se with your real address
$ RID=$(curl -s "https://blast.ncbi.nlm.nih.gov/blast/Blast.cgi" \
$   --data "CMD=Put&PROGRAM=blastp&DATABASE=swissprot&QUERY=P04637&FORMAT_TYPE=Text&email=your@email.se" \
$   | grep -o "RID = [A-Z0-9]*" | awk '{print $3}')
$ echo "RID: $RID" && sleep 45
# Retrieve results
$ curl -s "https://blast.ncbi.nlm.nih.gov/blast/Blast.cgi" \
$   --data "CMD=Get&RID=${RID}&FORMAT_TYPE=Text" > TP53_blastp_swissprot.txt
$ grep "^>" TP53_blastp_swissprot.txt | head -10
```

Part D — Local BLAST via Slurm

```
$ module load blast+/2.14.0
# Write a Slurm job script (see handout for full script)
$ sbatch blast_tp53.sh
$ squeue -u $USER                                  # Monitor job
# Once complete — tabular output (fmt 6): qseqid sseqid pident length mismatch gapopen...
$ head -20 TP53_blastp_local.txt
$ sort -k11 -n TP53_blastp_local.txt | head -20   # Sort by E-value
$ awk '$11 < 1e-10' TP53_blastp_local.txt | wc -l # How many strong hits?
$ git add . && git commit -m "lecture03: local BLAST results via Slurm"
```

What You Should Know After This Session

- ✓ Explain what homology means and distinguish orthologues from paralogues
- ✓ Describe the seed-and-extend strategy of BLAST at a conceptual level
- ✓ Explain what an E-value is and interpret one correctly, including the effect of database size
- ✓ Distinguish bit score from E-value and explain when bit score is more appropriate
- ✓ Select the correct BLAST programme for a given query and database type
- ✓ Interpret a BLAST results table: coverage, % identity, E-value, bit score
- ✓ Submit a BLAST job via the NCBI API from the command line and retrieve results
- ✓ Write a Slurm job script for local BLAST on HPC2N and interpret tabular output
- ✓ Commit BLAST inputs, scripts, and outputs to Git with meaningful commit messages